EBES: Easy Benchmarking for Event Sequences

Dmitry Osin
Skolkovo Institute of Science and Technology
Moscow, Russia
d.osin@skoltech.ru

Igor Udovichenko
Skolkovo Institute of Science and Technology
Moscow, Russia
Vega Institute Foundation
Moscow, Russia
i.udovichenko@skoltech.ru

Egor Shvetsov
Skolkovo Institute of Science and Technology
Moscow, Russia
e.shvetsov@skoltech.ru

Viktor Moskvoretskii
Skolkovo Institute of Science and Technology
Moscow, Russia
HSE University
Moscow, Russia
v.moskvoretskii@skoltech.ru

Evgeny Burnaev
Skolkovo Institute of Science and Technology
Moscow, Russia
Artificial Intelligence Research Institute
Moscow, Russia
e.burnaev@skoltech.ru

arXiv:2410.03399v2 [cs.LG] 25 Feb 2025

Abstract

Event Sequences (**EvS**) refer to sequential data characterized by irregular sampling intervals and a mix of categorical and numerical features. Accurate classification of these sequences is crucial for various real-life applications, including healthcare, finance, and user interaction. Despite the popularity of the **EvS** classification task, there is currently no standardized benchmark or rigorous evaluation protocol. This lack of standardization makes it difficult to compare results across studies, which can result in unreliable conclusions and hinder progress in the field. To address this gap, we present **EBES**, a comprehensive benchmark for **EvS** classification with sequence-level targets. EBES features standardized evaluation scenarios and protocols, along with an open-source PyTorch library[1] that implements 9 modern models. Additionally, it includes the largest collection of **EvS** datasets, featuring 10 curated datasets, including a novel synthetic dataset and real-world data with the largest publicly available banking dataset. The library offers user-friendly interfaces for integrating new methods and datasets. Our benchmarking results highlight the unique properties of **EvS** compared to other sequential data types, provide a performance ranking of modern models—with GRU-based models achieving the best results—and reveal the challenges associated with robust **EvS** learning. The goal of EBES is to facilitate reproducible research, expedite progress in the field, and increase the real-world impact of **EvS** classification techniques.

1 Introduction

The world in which we live is constantly changing [24]. We continuously collect and analyze data to understand and navigate this dynamic environment. This ongoing data collection helps capture the evolving nature of reality and can be captured in sequential datasets, which can be further analyzed or used for modeling.

Sequential data encompass a wide array of formats, from texts and videos to financial transaction logs and physical measurements. In this work, we focus on a specific subset of sequential data, termed **Event Sequences (EvS)**. These sequences consist of data points, each of which can be described by both categorical and numerical attributes. Every data point in an **EvS** is linked to a specific timestamp, and these points are arranged into sequences according to their timestamps. This specific structure is prevalent across various domains, including but not limited to healthcare [16, 31], ecology [11], e-commerce [43], and finance [6, 14].

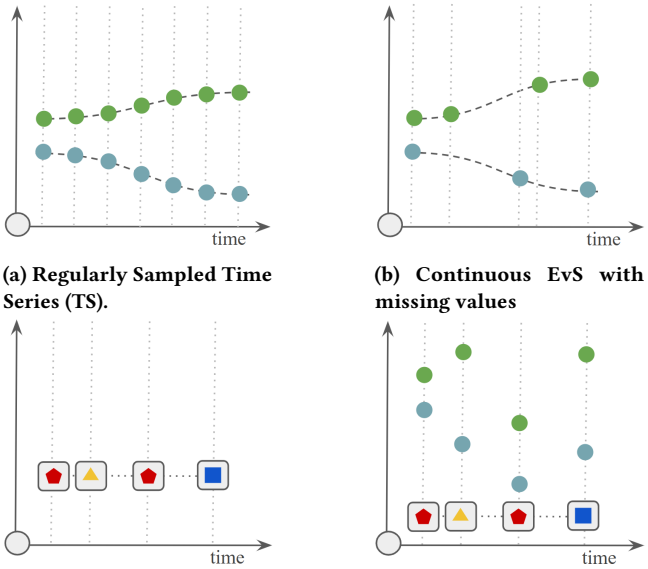
In the real world, many important problems involve **EvS** classification, where the goal is to attribute a target label to entire sequences rather than individual points. This type of classification is crucial in various domains, such as mortality prediction [21], churn prediction [20], predictive business process monitoring [28], and fraud detection [39].

Despite its popularity **EvS** classification lacks established benchmark and rigorous evaluation protocol. It leads to studies reporting inconsistent performance metrics [9, 19, 25, 34], making it hard to compare classification models. Manual hyperparameter tuning may lead to test leakage and biased estimation of performance [26].

To address the mentioned problems we present EBES, a benchmark tailored for **EvS** classification. In summary, our contributions are as follows:

(1) A rigorous evaluation protocol with statistically proven results and a curated list of datasets specifically designed for the classification of **EvS**.
(2) An open-source PyTorch library that implements the described evaluation protocol. This library is user-friendly and features unified interfaces, enabling the seamless addition of new methods and datasets in a plug-in fashion, without the need for individual adaptations.
(3) Evaluation results highlighting the need to treat **EvS** separately from closely related types of sequential data such as time series.
(4) A comprehensive empirical analysis that provides valuable insights into the properties of real-world **EvS** datasets and models developed for sequential data.

[1]Code is available at https://github.com/On-Point-RND/EBES



(a) Regularly Sampled Time Series (TS).

(b) Continuous EvS with missing values

(c) A stream of discrete events, usually, modeled by Temporal Point Process (TPP).

(d) Discrete EvS with 2 numerical and 1 categorical features.

Figure 1: Categorization of sequential data. Green and blue dots indicate numerical features, while different shapes denote categorical features. For TS and Continuous EvS, an underlying process is present. For Discrete EvS, no underlying process exists, making interpolation between neighboring points meaningless.

The paper is organized as follows. We formally define **EvS** classification in Section 2. The evaluation methodology is described in detail in Section 3. In Section 4, we list the datasets and methods selected for the benchmark. We present an empirical study of our benchmark in Section 5.

2 Background

Definition. Sequential data classification involves datasets comprising pairs (S_i, y_i), where S_i represents a sequence and y_i is the classification target attributed to the entire sequence. Each sequence $S_i = \{(x_i^j, t_i^j)\}_{j=1}^{n_i}$ consists of feature sets x_i^j and corresponding timestamps t_i^j. The nature of these sequences can be categorized based on the regularity of timestamps and the type of features (see Figure 1):

- **Time Series:** When timestamps t_i^j are *regular* and features x_i^j are exclusively numerical, the sequence is classified as a time series.
- **Streams of discrete events:** When timestamps t_i^j are *irregular* and features x_i^j are single categorical values, the sequence is modeled as a stream of discrete events, typically using Temporal Point Processes (TPP).
- **Event Sequences:** Our focus is on event sequences, characterized by *irregular* timestamps and diverse feature types. These sequences can be further subdivided:

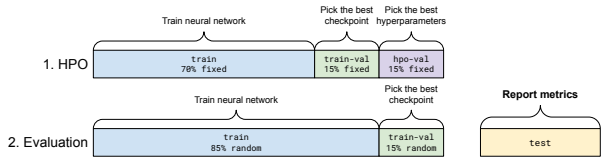


Figure 2: Data splits and their usage in our evaluation procedure. For each seed, the training sample is randomly divided into `train`, `train-val`, and `hpo-val` sets, while the `test` set is separated only once during data preprocessing.

  - **Continuous Event Sequences:** If all features x_i^j are snapshots of an underlying continuous process, the data is termed a **continuous EvS**. These sequences often require interpolation techniques for analysis [9, 33, 34].
  - **Discrete Event Sequences:** If the interpolation between neighboring points in a sequence is meaningless (for example, card transactions), the data is classified as a **discrete EvS**.

3 Methodology

In this section, we describe the evaluation methodology. The goal is to enable a robust and rigorous comparison of methods, ensuring accurate conclusions.

3.1 Data prepossessing

In our work, we follow common practices whenever possible to prevent data preprocessing from affecting model evaluation. For ease of extensibility, we convert all datasets into a single format and release scripts that perform the conversion. Our data preprocessing includes:

- Applying a logarithm to fat-tailed variables, which are selected manually according to [4];
- Rescaling time points to ensure the time range of all sequences falls within $[0, 1]$;
- For missing values, we propagate them forward for datasets with continuous **EvS** based on results from [8], and impute with constants for others.

EBES's data preprocessing is highly flexible and customizable, as it is defined by a single YAML config.

In the preprocessing step, we split the data into full-train and test subsets. For datasets with an established test set, we use that. For datasets without a predefined test set, we employ time-based splitting when possible, reserving the last 20% of sequences for testing. When time-based splitting is not feasible, we randomly select 20% of the sequences for testing.

3.2 Hyperparameter optimization

Hyperparameters are a fundamental aspect of machine learning that directly impacts model performance. However, the procedure of hyperparameter tuning is rarely described. Therefore, this becomes a source of non-reproducibility [3, 18]. Moreover, manual hyperparameter tuning can lead to the leakage of the test set into the training procedure and performance [26]; at the same time, testing different hyperparameter values is necessary to find a model that generalizes well [18].

Hyperparameter optimization (HPO) is at the core of our methodology. For HPO we use Optuna [2] Tree-structured Parzen Estimator (TPE), as its effectiveness has been proven on real-world tasks [23]. For each model and each dataset we allocate an HPO budget of 210 train runs capping the total compute at 18 NVidia A100 GPU-days. The best hyperparameters are used to evaluate the model. We emphasize that the procedure of HPO is fully formalized ensuring the test set is not leaked implicitly into the HPO procedure. To train the model during HPO runs, we use only the full-train subset. This subset is split into three parts:

- `train`: Used for training the models.
- `train-val`: Used for the early stopping procedure and checkpoint selection. Training is stopped if the target metric does not improve after several epochs and exceeds the patience limit.
- `hpo-val`: Used for model evaluation during HPO.

These splits remain fixed during the HPO. Both `train-val` and `hpo-val` take 15% from the initial train dataset. See Figure 2 for clarification.

3.3 Model evaluation

For model evaluation, we retrain the model with the best hyperparameters 20 times, each time using a different random seed. For each run, we *randomly* split the full-train data subset into a `train` set (85%) and a `train-val` set (15%). The `train` set is used to train the neural network weights, while the `train-val` set is used for early stopping and selecting the best checkpoint. This approach is known in the literature as Monte Carlo cross-validation [40].

We report the mean and standard deviation of the target metric computed on the test set across these 20 runs. To rank the models rigorously we perform Mann-Whitney tests [27] with Benjamini–Hochberg correction [7].

4 Benchmark

In this section, we describe the datasets and the methods we selected of our benchmark.

4.1 Datasets

We curated datasets from each domain: Time Series, Continuous **EvS** and Discrete **EvS** to illustrate distinctions between them.

- **Discrete EvS:** We selected three widely-used datasets: AGE, Retail, and Taobao, based on previous studies [4, 6, 30, 34, 36]. We also introduce two datasets to the broad community. The first is the BPI17 dataset [37], widely used in the subject field yet not well explored in general works on event sequences modeling. The dataset comprises the logs of the business process of negotiating a loan request. The second dataset is one of the largest event sequence datasets, MBD [14]. It contains multi-modal data about the bank clients. We take only the transaction log from the dataset.
- **Continuous EvS:** We utilized two medical datasets, PhysioNet 2012 and MIMIC-III. Additionally, we included a synthetic Pendulum dataset, which was specifically designed not only as an example of continuous **EvS** but also to validate the importance of temporal dynamics and assess how models capture sequential properties.
- **Time Series:** To highlight the distinctions of **EvS** we also chose one multivariate dataset (ArabicDigits) and one univariate dataset (ElectricDevices).

Our dataset selection spans various domains, including two medical, three banking, one retail, one synthetic, and one business process mining dataset, reflecting diverse complexities and difficulties. We also included datasets of varying sizes to address scalability challenges. All datasets are open-access to promote reproducibility and collaboration and we welcome contributions from other domains to enrich our collection. Adding new datasets is straightforward: simply convert the dataset into the correct format and prepare a YAML configuration file following the provided examples. Table 1 presents statistics for each dataset, and Appendix B provides detailed descriptions.

4.2 Models

We have carefully curated diverse models and approaches previously applied to **EvS** classification tasks. In this section, we first list models from two broad categories: general-purpose sequential data models and models specialized for certain domains. Following this, we describe the common structure shared by all models.

4.2.1 General Sequential Data Models. This category includes widely-used architectures designed for handling sequential data. Specifically, we include **GRU** [10], **Mamba** [17], and **Transformer** [38], all trained in a supervised manner. These models are strong baselines due to their proven effectiveness across sequential data domains. As a simple baseline, we evaluate a multi-layer perceptron (**MLP**) — a 3-layer fully-connected neural network that takes aggregated events in a sequence as input.

4.2.2 Models Specialized for Certain Domain. The second category comprises methods designed explicitly for certain domains of sequential data. Among these, **CoLES** [4] and **MLEM** [29] leverage contrastive learning or a combination of contrastive and generative strategies for unsupervised pre-training, followed by fine-tuning on downstream classification tasks. In addition, we include models tailored for continuous **EvS** data. The **mTAND** architecture, introduced in [34], is an attention-based model designed to interpolate and classify irregularly sampled time series with missing values. Building on this, **PrimeNet** [9] extends mTAND by incorporating time-sensitive contrastive learning and data reconstruction tasks during pre-training. To assess the performance of state-of-the-art methods in multivariate time series classification, we also include **ConvTran** [15]. Refer to Appendix A for a detailed model description.

4.2.3 Common Model Structure. Most selected methods share a common four-step structure, enabling systematic comparisons of design choices such as event time processing and sequence aggregation. This structure is reflected in the source code of EBES, making it easier to implement and test new models. Below, we outline these steps:

(1) **Preprocessing block.** This step transforms raw event sequences, described by categorical and real-valued features, into vector representations suitable for neural networks. The block encodes

Table 1: Statistics of sequential datasets used in our benchmark. The statistics are calculated on the train set if not specified otherwise.

Dataset	Domain	# classes	Class balance, %	Target	Category
ElectriDevices	Time Series	7	27 / 25 / 17 / 10 / 8 / 8 / 6	Device type	Engineering
ArabicDigits	Time Series	10	balanced	Digit	Motion
PhysioNet 2012	Continuous **EvS**	2	86 / 14	Mortality	Medical
MIMIC-III	Continuous **EvS**	2	90 / 10	Mortality	Medical
Pendulum	Continuous **EvS**	10	balanced	Air resistance	Physical (synth.)
AGE	Discrete **EvS**	4	25 / 25 / 25 / 25	Age group	Transactions
Retail	Discrete **EvS**	4	27 / 21 / 27 / 24	Age group	Transactions
MBD	Discrete **EvS**	4 × 2	99.7 ± 0.2 / 0.3 ± 0.2	Purchase items	Transactions
Taobao	Discrete **EvS**	2	43 / 57	Purchase event	E-commerce
BPI17	Discrete **EvS**	2	70 / 30	Offer acceptance	Business process mining
	# seq. (train / test)	# points (train / test)	# points per seq. (mean ± std)	# cat. features	# num. features
ElectriDevices	9k / 8k	857k / 740k	96 ± 0	0	1
ArabicDigits	7k / 2k	263k / 87k	40 ± 9	0	13
PhysioNet 2012	4k / 4k	299k / 299k	75 ± 23	3	38
MIMIC-III	45k / 11k	2.7m / 657k	58 ± 93	1	10
Pendulum	80k / 20k	2.5m / 631k	32 ± 9	0	2
AGE	24k / 6k	21m / 5.3m	881 ± 125	1	1
Retail	319k / 80k	37m / 9.1m	114 ± 103	7	9
MBD	7.4m / 1.8m	156m / 39m	21 ± 435	11	1
Taobao	18k / 9k	5.1m / 2.8m	280 ± 387	2	0
BPI17	34k / 9k	444k / 119k	13 ± 9	7	5

categorical features using embedding layers. For numerical features, it applies batch normalization and projects each feature individually into a larger space using a linear transformation. Two models explicitly process timestamps through specialized mechanisms. To ensure that all models can effectively utilize temporal information, regardless of their inherent design, timestamps can be concatenated with other event features in various ways: as absolute times, as time differences between consecutive events, or not concatenated at all. This approach treats time as just another feature describing an event.

(2) **Encoder block.** The encoder block maps the sequence of event vectors into another sequence of latent representations. The architectures differ most significantly here, with each model employing its unique mechanism for capturing temporal dependencies.

(3) **Aggregation of latent representations.** The block aggregates a sequence of latent vectors into a single vector. Common strategies include taking the last vector in the latent sequence or computing an average across the entire sequence.

(4) **Classification Head.** The final block predicts class logits from the aggregated representation, completing the classification process.

This unified structure facilitates a fair comparison of different design choices across models and datasets, highlighting the impact of specific architectural decisions on **EvS** classification performance.

5 Results and Analysis

5.1 Main result

The results of **Final Evaluation** from Section 3.2 are presented in Table 2, where methods are ranked from top to least performing. Alongside the mean performance, we report the method's rank as a superscript. We performed pairwise Mann–Whitney U test [27] with Benjamini–Hochberg correction [7], methods with no significant performance difference ($p > 0.01$) share the same superscript. We make the following observations.

GRU-based models dominate EvS classification. The top three performing methods are all based on GRU with varying pre-training strategies. This is intriguing, as other domains of sequential modeling are often dominated by attention-based or convolution-based architectures. CoLES demonstrates improved metrics on tasks where the target is a characteristic of the observed sequence, such as Age, Pendulum, and Retail. However, on datasets where the target is related to future events, such as MBD, Taobao, BPI17, PhysioNet, and MIMIC-III, pre-training does not provide a significant boost. This is due to CoLES' pre-training procedure, which treats all subsequences of a sequence as belonging to the same class as the full sequence. Even though MLEM utilizes pre-trained CoLES components, it performs similarly to CoLES and does not outperform both GRU and CoLES simultaneously on any dataset.

Transformer and Mamba are next in the ranking, suggesting that these architectures may be less suited for **EvS** classification, although they dominate the text domain, which is also sequential data.

Table 2: Model performance obtained using EBES. Results are averaged over 20 runs, with the best hyperparameters determined through HPO. Statistically indistinguishable ($p > 0.01$) results share the same superscripts, indicating the method's rank for each dataset. The best-performing methods for each dataset are highlighted. Methods are sorted according to their average rank across all datasets. Note: 4/20 runs of mTAND on the Pendulum dataset were excluded due to non-convergence (<20% accuracy). Number of learnable parameters presented in Table 6.

Category	Discrete **EvS**					Continuous **EvS**			Time Series	
Dataset	**MBD**	**Retail**	**Age**	**Taobao**	**BPI17**	**PhysioNet2012**	**MIMIC-III**	**Pendulum**	**ArabicDigits**	**ElectricDevices**
Metric	Mean ROC AUC	Accuracy	Accuracy	ROC AUC	ROC AUC	ROC AUC	ROC AUC	Accuracy	Accuracy	Accuracy
CoLES	0.826 ± 0.001^{2}	$\mathbf{0.553 \pm 0.002^{1}}$	$\mathbf{0.634 \pm 0.005^{1}}$	$\mathbf{0.713 \pm 0.002^{1}}$	$0.742 \pm 0.010^{3,4}$	$0.840 \pm 0.004^{2,3}$	$\mathbf{0.902 \pm 0.001^{1}}$	0.740 ± 0.013^{2}	$\mathbf{0.983 \pm 0.004^{1,2}}$	$\mathbf{0.729 \pm 0.019^{1,2}}$
GRU	$\mathbf{0.827 \pm 0.001^{1}}$	0.543 ± 0.002^{2}	0.626 ± 0.004^{2}	$\mathbf{0.713 \pm 0.004^{1}}$	$\mathbf{0.754 \pm 0.004^{1}}$	$\mathbf{0.846 \pm 0.004^{1}}$	$\mathbf{0.901 \pm 0.002^{1}}$	0.683 ± 0.031^{3}	0.975 ± 0.003^{4}	$\mathbf{0.741 \pm 0.013^{1}}$
MLEM	0.824 ± 0.001^{3}	0.544 ± 0.002^{2}	$\mathbf{0.634 \pm 0.003^{1}}$	$\mathbf{0.713 \pm 0.004^{1}}$	$\mathbf{0.753 \pm 0.005^{1,2}}$	$\mathbf{0.846 \pm 0.007^{1}}$	0.899 ± 0.002^{2}	0.676 ± 0.017^{3}	0.978 ± 0.002^{3}	$\mathbf{0.736 \pm 0.014^{1}}$
Transformer	0.821 ± 0.002^{4}	$0.536 \pm 0.006^{3,4}$	0.621 ± 0.006^{2}	$0.692 \pm 0.013^{3,4}$	$\mathbf{0.749 \pm 0.006^{1,2,3}}$	$0.838 \pm 0.008^{2,3,4}$	0.894 ± 0.002^{3}	0.658 ± 0.019^{4}	$\mathbf{0.986 \pm 0.004^{1,2}}$	0.710 ± 0.024^{2}
Mamba	0.820 ± 0.003^{4}	0.538 ± 0.003^{3}	0.609 ± 0.006^{3}	$0.693 \pm 0.023^{2,3}$	$0.737 \pm 0.012^{4,5}$	$0.835 \pm 0.006^{3,4}$	0.895 ± 0.002^{3}	0.687 ± 0.017^{3}	0.983 ± 0.005^{2}	0.716 ± 0.022^{2}
ConvTran	0.816 ± 0.002^{5}	0.534 ± 0.005^{4}	0.603 ± 0.006^{4}	0.703 ± 0.009^{2}	$0.748 \pm 0.006^{2,3}$	$0.837 \pm 0.006^{2,3,4}$	$0.892 \pm 0.005^{3,4}$	$0.674 \pm 0.028^{3,4}$	$\mathbf{0.986 \pm 0.003^{1}}$	0.711 ± 0.019^{2}
mTAND	0.798 ± 0.002^{7}	0.519 ± 0.003^{6}	0.582 ± 0.009^{5}	0.672 ± 0.010^{5}	0.738 ± 0.005^{4}	0.841 ± 0.005^{2}	$0.888 \pm 0.003^{4,5}$	$\mathbf{0.777 \pm 0.031^{1}}*$	0.951 ± 0.010^{5}	0.631 ± 0.019^{3}
PrimeNet	0.780 ± 0.006^{8}	0.521 ± 0.003^{6}	0.583 ± 0.011^{5}	0.681 ± 0.010^{4}	0.730 ± 0.006^{5}	$0.839 \pm 0.004^{2,3,4}$	0.887 ± 0.004^{5}	0.600 ± 0.026^{5}	0.958 ± 0.009^{5}	0.636 ± 0.016^{3}
MLP	0.809 ± 0.001^{6}	0.526 ± 0.002^{5}	0.581 ± 0.007^{5}	0.659 ± 0.035^{5}	0.737 ± 0.004^{4}	0.835 ± 0.004^{4}	0.881 ± 0.001^{6}	0.186 ± 0.006^{6}	0.760 ± 0.011^{6}	0.437 ± 0.019^{4}

Success in TS classification doesn't guarantee EvS classification performance. mTAND [34] excels on the Pendulum dataset due to its architecture tailored for modeling the time component, making it well-suited for datasets like Pendulum. Similarly, ConvTran performs best on the Multivariate TS dataset, ArabicDigits, for which it was specifically designed. On this dataset, other models with attention mechanisms, such as Transformer, also perform well. However, both mTAND and ConvTran struggle on other datasets, indicating that these methods may not be as effective for general event sequence classification. This suggests that state-of-the-art models designed for time series analysis do not necessarily perform well on event sequence classification tasks, underscoring the need to treat **EvS** as a distinct domain.

Notably, even though mTAND was explicitly designed for PhysioNet2012 and MIMIC-III, and ConvTran was theoretically better suited for ElectricDevices, both were outperformed by **EvS**-specific models. This asymmetry suggests an intriguing direction: rather than attempting to adapt TS methods to **EvS** tasks, it may be worthwhile to explore the reverse—applying **EvS** models to TS problems.

MLP's performance suggests effective EvS classification with aggregated statistics. The MLP performs relatively well, typically within 5% of the top-performing method on most real-world datasets. This suggests that **EvS** classification can be effectively carried out using aggregated statistics along temporal dimensions, a practice commonly employed in industrial applications with boosting models [1]. The difference in performance between MLP and mTAND on the Pendulum dataset further supports this idea, since we cannot apply such aggregation approach to this dataset.

The PhysioNet2012 dataset's effectiveness for EvS classification evaluation is questionable. Despite its widespread use in modeling irregularly sampled time series, all methods, including the MLP, achieve closely ranked results, suggesting limited ability to distinguish model performance. We include this dataset precisely because of its prevalence in the literature, aiming to demonstrate its shortcomings and highlight the need for more discriminative datasets in **EvS** classification tasks.

5.2 The Role of Sequence Order and Time in EvS classification

To understand how important the order of events and their timestamps are in **EvS** classification, we conducted three experiments: 1) we tested models trained on original sequences with shuffled event orders; 2) we tested models trained on original sequences with random timestamps; and 3) to check whether temporal information matters at all, we removed timestamps and retrained models on shuffled sequences, ensuring the model had no information about temporal structure.

5.2.1 Testing on Permuted Sequences. We evaluated pre-trained models on perturbed sequences *without fine-tuning*. Missing values were filled prior to shuffling, and time was added as a numerical feature before shuffling. For all runs, the last events were kept in their original positions, as some models use the last hidden state in the aggregation step. Results are presented in Table 3. We made two key observations:

- The datasets can be clearly divided into two groups based on the performance drop. The group with the largest drop includes time series datasets (ArabicDigits and ElectricDevices) and a synthetic dataset designed to evaluate the ability to capture temporal structure (Pendulum). The second group consists of real-world **EvS** data, where the performance drop is relatively small, indicating that sequence order is not as critical for **EvS** classification as it might seem to be.
- Models specifically designed for time series exhibit a larger drop in performance across all datasets compared to **EvS** models.

This indicates that capturing the properties of **EvS** requires specific models, further supporting the importance of **EvS** as a distinct domain.

The Transformer model experienced a minimal drop on **EvS** datasets due to its attention mechanism. Notably, the drop is zero on datasets where positional embedding (PE) was not used (the use of PE was determined during HPO). The MLP model did not experience any performance drop because sequence order is inherently unimportant for aggregation. Notably, the MBD dataset did not experience a significant drop with most methods.

Table 3: Testing on Permuted Sequences. Models were trained on non-permuted data; only the test set was permuted. We report performance difference relative to metrics obtained on not permuted sequences. Only values with statistically significant difference ($p < 0.01$) in performance are highlighted.

Category	Discrete **EvS**					Continuous **EvS**			Time Series	
Dataset	**MBD**	**Retail**	**Age**	**Taobao**	**BPI17**	**PhysioNet2012**	**MIMIC-III**	**Pendulum**	**ArabicDigits**	**ElectricDevices**
Metric	Mean ROC AUC	Accuracy	Accuracy	ROC AUC	ROC AUC	ROC AUC	ROC AUC	Accuracy	Accuracy	Accuracy
CoLES	−0.09%	−1.57%	−1.63%	−0.49%	−4.66%	−2.36%	−1.86%	−84.49%	−33.86%	−68.79%
GRU	−0.10%	−2.25%	−1.15%	−0.67%	−4.46%	−1.49%	−4.24%	−76.09%	−46.88%	−69.46%
MLEM	−0.30%	−2.57%	−1.52%	−0.89%	−3.80%	−1.71%	−1.43%	−81.84%	−37.81%	−65.17%
Transformer	−0.00%	−0.09%	−0.00%	−0.05%	−0.00%	0.03%	−0.00%	−0.00%	−15.12%	−25.26%
Mamba	−0.06%	−2.44%	−1.20%	−0.00%	−9.56%	−0.65%	−3.04%	−82.14%	−53.37%	−54.18%
ConvTran	−7.28%	−29.02%	−9.55%	−4.51%	−17.04%	−0.47%	−8.21%	−77.61%	−60.45%	−68.66%
mTAND	−5.05%	−28.09%	−8.95%	−4.13%	−9.07%	−4.13%	−5.05%	−82.57%	−59.12%	−56.04%
PrimeNet	−4.08%	−26.41%	−7.82%	−2.12%	−4.73%	−3.95%	−3.72%	−75.88%	−53.38%	−54.38%
MLP	−0.00%	−0.00%	−0.00%	−0.00%	−0.00%	−0.00%	−0.00%	−0.00%	−0.00%	−0.00%

Table 4: Testing on Random Timestamps. Models were trained on original data; only the test set has random timestamps. We report performance difference relative to metrics obtained on original sequences. Only values with statistically significant difference ($p < 0.01$) in performance are highlighted.

Category	Discrete **EvS**					Continuous **EvS**			Time Series	
Dataset	**MBD**	**Retail**	**Age**	**Taobao**	**BPI17**	**PhysioNet2012**	**MIMIC-III**	**Pendulum**	**ArabicDigits**	**ElectricDevices**
Metric	Mean ROC AUC	Accuracy	Accuracy	ROC AUC	ROC AUC	ROC AUC	ROC AUC	Accuracy	Accuracy	Accuracy
PrimeNet	−0.72%	−0.07%	−0.12%	−0.15%	−0.30%	0.09%	−0.40%	−66.34%	−28.86%	−5.62%
mTAND	−0.45%	−0.01%	−0.06%	−0.91%	−0.00%	−0.08%	−0.23%	−56.79%	−7.44%	−6.11%

Table 5: Training on Permuted Sequences without Timestamps. The GRU model with the best hyperparameters had the time feature removed and was then trained from scratch in two settings: *with* and *without* permuting both the training and test sequences. We report performance difference relative to metrics obtained on original sequences. Only values with statistically significant difference ($p < 0.01$) in performance are highlighted.

Category	Discrete **EvS**					Continuous **EvS**			Time Series	
Dataset	**MBD**	**Retail**	**Age**	**Taobao**	**BPI17**	**PhysioNet2012**	**MIMIC-III**	**Pendulum**	**ArabicDigits**	**ElectricDevices**
Metric	Mean ROC AUC	Accuracy	Accuracy	ROC AUC	ROC AUC	ROC AUC	ROC AUC	Accuracy	Accuracy	Accuracy
GRU w/o time	−0.89%	−0.00%	−0.44%	−3.85%	−0.00%	−0.00%	−0.27%	−59.43%	0.04%	−0.00%
GRU w/o time w/ perm.	−0.96%	0.50%	0.62%	−1.54%	−0.45%	−0.22%	−1.25%	−63.87%	−1.28%	−16.00%

5.2.2 Testing on Random Timestamps. Two methods are specifically designed to model the time component in our selection of methods: mTAND [34] and PrimeNet [9]. We evaluated them on test data with noisy timestamps, where the original timestamps were replaced with random values sorted in ascending order. The results are presented in Table 4. While time is important for these models on the ArabicDigits, ElectricDevices and synthetic Pendulum dataset, it did not contribute significantly to the other datasets.

We conclude that methods specifically designed to work with time do not effectively capture temporal dependencies on real-world **EvS** datasets. This emphasizes the importance of developing or testing new methods on **EvS** that can model the time component on real-world datasets.

5.2.3 Training on Permuted Sequences without Timestamps. The second experiment further analyzed datasets to determine if order information is important or if sequences can be treated as a **"bag of words".** We selected the GRU with the best hyperparameters for each dataset and removed time completely from the model inputs. The model was then trained from scratch under two conditions: **with** and **without** permuting both the training and test sequences. The results are presented in Table 5.

We observed that for some real-world datasets, the performance drop was not statistically significant. We speculate that such permutation could even serve as a form of data augmentation, since in some cases mean metrics increased with permutation. This phenomenon was observed in datasets such as Retail, Age, BPI17, and Physionet2012.

On the other hand, datasets like MBD and Taobao showed performance declines when the time feature was removed. However, permuting the sequences on top of that did not cause a significant further drop. This suggests that for these datasets, the time feature is more important than the sequence order itself.

Finally, datasets such as MIMIC-III, Pendulum, ArabicDigits, and ElectricDevices demonstrated a clear performance drop, underscoring the importance of sequence order for the GRU model on those datasets.

From both experiments, we conclude that sequence order is important for **EvS** classification, but it is less critical than expected for real-world datasets and varies from dataset to dataset.

5.3 HPO analysis

5.3.1 Assessing Architecture Design Choices. This section evaluates the impact of key architecture design choices—aggregation, normalization, learning rates, and time features—across various models and datasets, providing insights from our hyperparameter optimization (HPO) procedure.

First, we examine the effect of temporal aggregation methods, comparing the mean of all hidden states to the last hidden state. Results show that the choice of aggregation significantly impacts performance. CoLES and MLEM consistently benefit from using the last hidden state across all datasets, as do Physionet2012 and MIMIC-III. In contrast, Mamba performs better with the mean hidden state on all datasets except Physionet2012, while other models exhibit individual preferences. Detailed results are provided in Table 8.

Second, we analyze the impact of batch normalization on numerical features. Results indicate that batch normalization improves performance for nearly all methods and datasets, with the exception of Pendulum and ArabicDigits. See Table 9 for further details.

We also investigate the role of time features. Table 7 demonstrates that incorporating time significantly enhances performance for MBD, Age, Taobao, MIMIC-III, and Pendulum datasets. However, methods such as Retail, BPI17, and ArabicDigits show no notable benefit. While these results highlight dataset-specific trends, they do not preclude the potential importance of time in other contexts, as alternative integration methods—unexplored in this study—may yield further improvements.

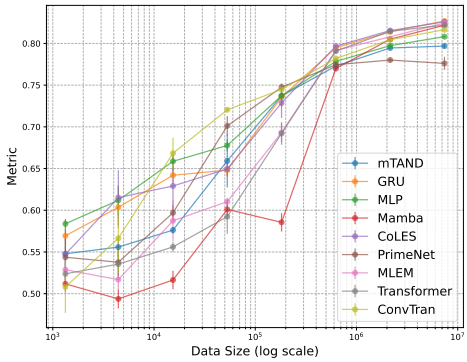
Finally, HPO evaluations consistently identify the learning rate as one of the most important hyperparameter across all runs, as summarized in Table 10. This underscores its critical role in optimizing model performance.

5.3.2 Dataset Analysis. In this section, we analyze datasets based on data from the **HPO** and **Final Evaluation** phases, exploring relationships between metrics from different data subsets. See Appendix E.

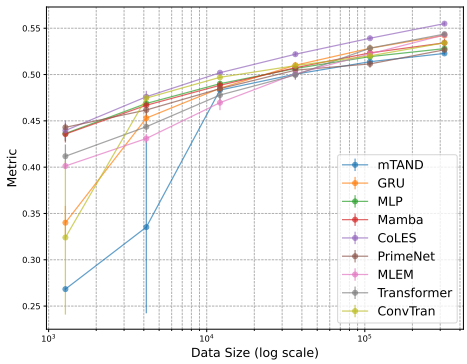
During the HPO step, we observe overfitting for most datasets, as `train` metrics increase while `train-val` metrics plateau, as seen in Figure 9 on the left. This supports the use of early stopping.

Metrics of `hpo-val` and `test` subsets (third column in Figure 9) are strongly correlated unless the test set is sampled out-of-time, as seen for the Taobao and BPI17 dataset. Here, `hpo-val` and `test` metrics lack a clear linear trend, but `train-val` and `hpo-val` metrics do, suggesting a distribution shift in the test set.

For most datasets, in the **Final Evaluation** phase (fourth column in Figure 9), validation and test set metrics exhibit a linear trend, except for PhysioNet2012, where different validation metrics attribute to similar test metrics. This supports our observations in Section 5.1, where results for most models are not statistically distinguishable for most methods on PhysioNet2012.



(a) MBD dataset



(b) Retail dataset

Figure 3: Data Scaling Results. We take models with the best hyperparameters and retrain them on subsets of varying sizes. The number of sequences is presented on a log scale. Standard deviation across 3 runs is indicated by vertical lines.

5.4 Data Scaling Results

To study the scaling properties of various models, we evaluated each model trained with different numbers of sequences. We focused on two biggest real-world dataset in our benchmark: Retail and MBD. We sampled different subsets, each containing progressively more data. Each model was trained from scratch on different-sized subsets with Monte Carlo cross-validation using three random seeds.

A common approach is to estimate model performance with a fixed data size. However, as seen in Figure 3, while all models improve with the growth of the data, their ranking does not stay the same, except for CoLES on the Retail Dataset, where it consistently demonstrates superior performance. With some data size, even MLP becomes a top performer. Most models, except for MLP, mTAND, and PrimeNet, converge to similar performance on the MBD dataset given a large data size. It is worth noting that for each dataset, we used the best hyperparameters found for each model when the dataset was at its full size.

The standard deviation, decreases as the data size increases and models perform very differently with smaller subsets. This makes evaluating model performance on relatively small datasets more prone to misleading results.

6 Related work

The UCR Time Series Archive [13], a widely used benchmark for time series classification, comprises 128 datasets designed to evaluate algorithms on univariate time series data. While this archive has been instrumental in advancing time series research, it does not address the unique challenges posed by **EvS** data, such as irregular intervals and heterogeneous feature types. Similarly, the `torchtime` package [12] extends the utility of UEA and UCR datasets by providing reproducible implementations for PyTorch. However, it remains focused on the classification of time series and does not cover the complexity of **EvS**.

The temporal point process (TPP) formalism is often used to model various types of **EvS**. EasyTPP [41] is a recent benchmark targeting streams of discrete events, offering a centralized repository for evaluating TPP models. However, EasyTPP focuses primarily on next-event prediction tasks, such as forecasting the time and type of the next event, rather than sequence-level classification. Many datasets in EasyTPP consist of only one categorical feature per event and a timestamp. Moreover, the majority of them lack established targets attributed to entire sequences, thus making them unsuitable for evaluating **EvS** classification methods.

Another closely related benchmark is HoTPP [22], which evaluates TPP models on long-term prediction tasks rather than next-event prediction. While HoTPP addresses a different aspect of event sequence modeling, it still does not focus on sequence-level classification, which is our primary goal.

7 Limitations and future work

We acknowledge that conducting a full hyperparameter optimization process requires substantial computational resources, which may not be available to all users. Additionally, using a random subset of the full training set as a proxy for the test set is not always the best method for estimating hyperparameters, as seen in Figure 10. The development of more efficient strategies for proper model evaluation and more robust procedures to address potential data shifts could be a promising direction for future research.

Our work focuses solely on one task—**EvS** classification while there are various tasks applied to **EvS**. We leave this for future work.

8 Conclusion

In this work, we introduced **EBES**, an open and comprehensive benchmark designed to enable standardized and transparent comparisons of models for **EvS** classification. By curating a diverse range of datasets and models, EBES provides a robust framework for evaluating various approaches' performance while also offering a user-friendly interface and a rich library for easy integration of new datasets and methods.

First, EBES establishes **EvS** classification as a distinct domain requiring specialized methods. Our analysis demonstrates that **EvS** has unique properties and behaviors that set it apart from related sequential data types like time series. This is supported by statistically proven results showing that models perform differently on **EvS** compared to other sequential data. In particular, GRU-based models dominate **EvS** classification, while time series-specific models perform worse. We also demonstrated that the importance of time and sequential structure varies across real-world datasets and tends to be lower than expected on **EvS** datasets compared to time series datasets. This variability underscores the necessity of developing or adapting models that inherently account for the time component in real-world scenarios rather than relying on methods initially designed for time series or discrete event streams.

We demonstrated how HPO analysis can reveal hidden trends in architecture design and datasets structure. For instance, results on the PhysioNet2012 dataset reveal a limited ability to distinguish model performance, suggesting that it may not be suitable for benchmarking **EvS**-specific methods. Additionally, the observed distribution shifts in out-of-time splits for datasets like Taobao and BPI17 emphasize the importance of accounting for real-world complexities during model validation and hyperparameter optimization.

The insights gained from EBES provide a foundation for future research, encouraging the development of novel architectures and techniques that address the unique challenges posed by event sequences. As researchers continue to explore this domain, we hope that EBES will catalyze innovation and guide the community toward more effective solutions for understanding and leveraging event sequence data.

References

[1] Lahcen ABIDAR, Dounia ZAIDOUNI, EL Ikram, and Abdeslam ENNOUAARY. 2023. Predicting Customer Segment Changes to Enhance Customer Retention: A Case Study for Online Retail using Machine Learning. *International Journal of Advanced Computer Science and Applications* 14, 7 (2023).

[2] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2623–2631.

[3] Christian Arnold, Luka Biedebach, Andreas Küpfer, and Marcel Neunhoeffer. 2023. The role of hyperparameters in machine learning models and how to tune them. *Political Science Research and Methods* (2023), 1–8.

[4] Dmitrii Babaev, Nikita Ovsov, Ivan Kireev, Maria Ivanova, Gleb Gusev, Ivan Nazarov, and Alexander Tuzhilin. 2022. Coles: contrastive learning for event sequences with self-supervision. In *Proceedings of the 2022 International Conference on Management of Data*. 1190–1199.

[5] Anthony Bagnall, Hoang Anh Dau, Jason Lines, Michael Flynn, James Large, Aaron Bostrom, Paul Southam, and Eamonn Keogh. 2018. The UEA multivariate time series classification archive. *arXiv preprint arXiv:1811.00075* (2018).

[6] Alexandra Bazarova, Maria Kovaleva, Ilya Kuleshov, Evgenia Romanenkova, Alexander Stepikin, Alexandr Yugay, Dzhambulat Mollaev, Ivan Kireev, Andrey Savchenko, and Alexey Zaytsev. 2024. Universal representations for financial transactional data: embracing local, global, and external contexts. *arXiv e-prints* (2024), arXiv–2404.

[7] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal statistical society: series B (Methodological)* 57, 1 (1995), 289–300.

[8] Zhengping Che, Sanjay Purushotham, Kyunghyun Cho, David Sontag, and Yan Liu. 2018. Recurrent neural networks for multivariate time series with missing values. *Scientific reports* 8, 1 (2018), 6085.

[9] Ranak Roy Chowdhury, Jiacheng Li, Xiyuan Zhang, Dezhi Hong, Rajesh K. Gupta, and Jingbo Shang. 2023. PrimeNet: Pre-training for Irregular Multivariate Time Series. *Proceedings of the AAAI Conference on Artificial Intelligence* 37, 6 (Jun. 2023), 7184–7192. doi:10.1609/aaai.v37i6.25876

[10] Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio. 2014. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555* (2014).

[11] James S Clark and Ottar N Bjørnstad. 2004. Population time series: process variability, observation errors, missing values, lags, and hidden states. *Ecology*

85, 11 (2004), 3140–3150.
[12] Philip Darke, Paolo Missier, and Jaume Bacardit. 2022. Benchmark time series data sets for PyTorch–the torchtime package. *arXiv preprint arXiv:2207.12503* (2022).
[13] Hoang Anh Dau, Anthony Bagnall, Kaveh Kamgar, Chin-Chia Michael Yeh, Yan Zhu, Shaghayegh Gharghabi, Chotirat Ann Ratanamahatana, and Eamonn Keogh. 2019. The UCR time series archive. *IEEE/CAA Journal of Automatica Sinica* 6, 6 (2019), 1293–1305.
[14] Mollaev Dzhambulat, Alexander Kostin, Postnova Maria, Ivan Karpukhin, Ivan A Kireev, Gleb Gusev, and Andrey Savchenko. 2024. Multimodal Banking Dataset: Understanding Client Needs through Event Sequences. arXiv:2409.17587 [cs.LG] https://arxiv.org/abs/2409.17587
[15] Navid Mohammadi Foumani, Chang Wei Tan, Geoffrey I Webb, and Mahsa Salehi. 2024. Improving position encoding of transformers for multivariate time series classification. *Data mining and knowledge discovery* 38, 1 (2024), 22–48.
[16] Ary L Goldberger, Luis AN Amaral, Leon Glass, Jeffrey M Hausdorff, Plamen Ch Ivanov, Roger G Mark, Joseph E Mietus, George B Moody, Chung-Kang Peng, and H Eugene Stanley. 2000. PhysioBank, PhysioToolkit, and PhysioNet: components of a new research resource for complex physiologic signals. *circulation* 101, 23 (2000), e215–e220.
[17] Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752* (2023).
[18] Odd Erik Gundersen, Kevin Coakley, Christine Kirkpatrick, and Yolanda Gil. 2022. Sources of irreproducibility in machine learning: A review. *arXiv preprint arXiv:2204.07610* (2022).
[19] Max Horn, Michael Moor, Christian Bock, Bastian Rieck, and Karsten Borgwardt. 2020. Set functions for time series. In *International Conference on Machine Learning*. PMLR, 4353–4363.
[20] Hemlata Jain, Ajay Khunteta, and Sumit Srivastava. 2021. Telecom churn prediction and used techniques, datasets and performance measures: a review. *Telecommunication Systems* 76 (2021), 613–630.
[21] Alistair EW Johnson, Tom J Pollard, Lu Shen, Li-wei H Lehman, Mengling Feng, Mohammad Ghassemi, Benjamin Moody, Peter Szolovits, Leo Anthony Celi, and Roger G Mark. 2016. MIMIC-III, a freely accessible critical care database. *Scientific data* 3, 1 (2016), 1–9.
[22] Ivan Karpukhin, Foma Shipilov, and Andrey Savchenko. 2024. HoTPP Benchmark: Are We Good at the Long Horizon Events Forecasting? *arXiv preprint arXiv:2406.14341* (2024).
[23] Nikita Klyuchnikov, Ilya Trofimov, Ekaterina Artemova, Mikhail Salnikov, Maxim Fedorov, Alexander Filippov, and Evgeny Burnaev. 2022. Nas-bench-nlp: neural architecture search benchmark for natural language processing. *IEEE Access* 10 (2022), 45736–45747.
[24] Diogenes Laertius. 1925. Lives of eminent philosophers, translated by RD Hicks. *Loeb Classical Library* 2 (1925), 408–423.
[25] Zekun Li, Shiyang Li, and Xifeng Yan. 2024. Time Series as Images: Vision Transformer for Irregularly Sampled Time Series. *Advances in Neural Information Processing Systems* 36 (2024).
[26] Michael A Lones. 2021. How to avoid machine learning pitfalls: a guide for academic researchers. *arXiv preprint arXiv:2108.02497* (2021).
[27] Henry B Mann and Donald R Whitney. 1947. On a test of whether one of two random variables is stochastically larger than the other. *The annals of mathematical statistics* (1947), 50–60.
[28] Alfonso Eduardo Márquez-Chamorro, Manuel Resinas, and Antonio Ruiz-Cortés. 2017. Predictive monitoring of business processes: a survey. *IEEE Transactions on Services Computing* 11, 6 (2017), 962–977.
[29] Viktor Moskvoretskii, Dmitry Osin, Egor Shvetsov, Igor Udovichenko, Maxim Zhelnin, Andrey Dukhovny, Anna Zhimerikina, and Evgeny Burnaev. 2024. MLEM: Generative and Contrastive Learning as Distinct Modalities for Event Sequences. *arXiv preprint arXiv:2401.15935* (2024).
[30] Viktor Moskvoretskii, Dmitry Osin, Egor Shvetsov, Igor Udovichenko, Maxim Zhelnin, Andrey Dukhovny, Anna Zhimerikina, Albert Efimov, and Evgeny Burnaev. 2024. Self-Supervised Learning in Event Sequences: A Comparative Study and Hybrid Approach of Generative Modeling and Contrastive Learning. arXiv:2401.15935 [cs.LG]
[31] Matthew A Reyna, Christopher S Josef, Russell Jeter, Supreeth P Shashikumar, M Brandon Westover, Shamim Nemati, Gari D Clifford, and Ashish Sharma. 2020. Early prediction of sepsis from clinical data: the PhysioNet/Computing in Cardiology Challenge 2019. *Critical care medicine* 48, 2 (2020), 210–217.
[32] Yulia Rubanova, Ricky TQ Chen, and David K Duvenaud. 2019. Latent ordinary differential equations for irregularly-sampled time series. *Advances in neural information processing systems* 32 (2019).
[33] Satya Narayan Shukla and Benjamin Marlin. 2018. Interpolation-Prediction Networks for Irregularly Sampled Time Series. In *International Conference on Learning Representations*.
[34] Satya Narayan Shukla and Benjamin Marlin. 2021. Multi-Time Attention Networks for Irregularly Sampled Time Series. In *International Conference on Learning Representations*. https://openreview.net/forum?id=4c0J6lwQ4_
[35] Sana Tonekaboni, Danny Eytan, and Anna Goldenberg. 2021. Unsupervised Representation Learning for Time Series with Temporal Neighborhood Coding. arXiv:2106.00750 [cs.LG]
[36] Igor Udovichenko, Egor Shvetsov, Denis Divitsky, Dmitry Osin, Ilya Trofimov, Ivan Sukharev, Anatoliy Glushenko, Dmitry Berestnev, and Evgeny Burnaev. 2024. SeqNAS: Neural Architecture Search for Event Sequence Classification. *IEEE Access* 12 (2024), 3898–3909. doi:10.1109/ACCESS.2024.3349497
[37] Boudewijn van Dongen. 2017. BPI Challenge 2017. doi:10.4121/uuid:5f3067df-f10b-45da-b98b-86ae4c7a310b
[38] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
[39] Yu Xie, Guanjun Liu, Chungang Yan, Changjun Jiang, and MengChu Zhou. 2022. Time-aware attention-based gated network for credit card fraud detection by extracting transactional behaviors. *IEEE Transactions on Computational Social Systems* (2022).
[40] Qing-Song Xu and Yi-Zeng Liang. 2001. Monte Carlo cross validation. *Chemometrics and Intelligent Laboratory Systems* 56, 1 (2001), 1–11.
[41] Siqiao Xue, Xiaoming Shi, Zhixuan Chu, Yan Wang, Hongyan Hao, Fan Zhou, Caigao Jiang, Chen Pan, James Y. Zhang, Qingsong Wen, Jun Zhou, and Hongyuan Mei. 2024. EasyTPP: Towards Open Benchmarking Temporal Point Processes. In *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2307.08097
[42] Jinsung Yoon, Daniel Jarrett, and Mihaela van der Schaar. 2019. Time-series Generative Adversarial Networks. In *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.), Vol. 32. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2019/file/c9efe5f26cd17ba6216bbe2a7d26d490-Paper.pdf
[43] Shiwei Zhao, Runze Wu, Jianrong Tao, Manhu Qu, Minghao Zhao, Changjie Fan, and Hongke Zhao. 2023. perCLTV: A general system for personalized customer lifetime value prediction in online games. *ACM Transactions on Information Systems* 41, 1 (2023), 1–29.

A Models description

GRU. We have chosen to use the GRU as one of our base models due to its proven effectiveness in encoding time-ordered sequences [4, 32, 35, 36, 42]. In recent study on neural architecture search [36]), authors demonstrated that architectures with RNN blocks tend to exhibit higher performance on average on EvS assesment task.

CoLES. The contrastive pretraining method for sequential data was proposed by [4]. We specifically focus on this method due to its superior performance compared to other contrastive approaches demonstrated in the work. CoLES learns to encode a sequence into a latent vector by bringing sub-sequences of the same sequence closer in the embedding space while pushing sub-sequences from different sequences further apart.

MLEM. The Multimodal Learning Event Model [30] is a recently proposed method for Event Sequences that unifies contrastive learning with generative modeling. It treats generative pre-training and contrastive learning as distinct modalities. First, a contrastive encoder is trained, followed by an encoder-decoder that learns latent states using reconstruction loss while aligning with contrastive embeddings to enhance the embedding information.

Transformer. As another base model, we added the transformer architecture [38], as attention-based architectures are one of the most common models in sequential modeling. We took the basic PyTorch implementation and provided three options for positional encoding (chosen in HPO): sinusoidal positional encoding added to features, sinusoidal positional encoding concatenated along the temporal dimension, learned positional embeddings, and no positional encoding at all.

MLP. The models applied 3 linear layers with the ReLU nonlinearity and dropouts in between to the aggregated embeddings obtained right after the preprocessing block. So effectively the model is just a basic MLP applied to aggregations. Models for **EvS** handles the sequential nature of data in a special way, ofthen considering the exact time intervals between the events, so we were interested in the performance of the model, that consciously discards the sequential nature of data.

Mamba. Mamba [17] is a recent state-space model (SSM) that has been designed for efficient handling of complex, long sequences. It incorporates selective state spaces to deliver top-notch performance across different modalities, including language, audio, and genomics, outperforming Transformers in some scenarios. For the best of our knowledge Mamba has not been applied to EvS classification previously, however, we believe that type of models worth of investigating.

mTAND. Authors in [34] proposed an architecture which learns an embedding of continuous-time values and utilizes an attention mechanism to produce a fixed-length representation of a time series. This procedure is specifically designed to deal with ISTS and has been shown to outperform numerous ordinary differential equations-based models such as Latent ODE and ODE-RNN [32].

PrimeNet. The method proposed in [9] also, falls under the category of self-supervised. It utilizes time-sensitive contrastive pretraining and enhances pretraining procedure with data reconstruction tasks to facilitate the usage of unlabeled data. Authors modify mTAN architecture by replacing an RNN block with Feature-Feature Attention.

ConvTran. We have chosen to use the ConvTran [15] due to its state-of-the-art performance in multivariate time series classification (MTSC) tasks. ConvTran combines novel position encoding techniques, specifically time Absolute Position Encoding (tAPE) and efficient Relative Position Encoding (eRPE), with convolution-based input encoding. This combination enhances the model's ability to capture both the temporal ordering and the data embedding of time series data effectively.

B Datasets Description

PhysioNet2012. dataset[2] was first intruduced in [16]. It includes multivariate time series data with 37 variables gathered from intensive-care unit (ICU) records. Each record contains measurements taken at irregular intervals during the first 48 hours of ICU admission. We used `set-a` as a train set and `set-b` as a test set. Both sets contain 4000 labeled sequences.

MIMIC-III. dataset[3] [21] consists of multivariate time series data featuring sparse and irregularly sampled physiological signals, collected at Beth Israel Deaconess Medical Center from 2001 to 2012. While we aimed to follow the general pipeline outlined in [33], we made several modifications to enhance the accuracy and reproducibility of our approach. Importantly, we did not alter the original problem statement: we excluded series that last less than 48 hours and used the first 48 hours of observations from the remaining series to predict in-hospital mortality. These adjustments were necessary to address certain issues and improve the overall robustness of our analysis.

Age. dataset[4] consists of 44M anonymized credit card transactions representing 50K individuals. The target is to predict the age group of a cardholder that made the transactions. The multiclass target label is known only for 30K records, and within this subset the labels are balanced. Each transaction includes the date, type, and amount being charged. The dataset was first introduced in scientific literature in work [4].

Retail. dataset[5] comprises 45.8M retail purchases from 400K clients, with the aim of predicting a client's age group based on their purchase history. Each purchase record includes details such as time, item category, the cose, and loyalty program points received. The age group information is available for all clients, and the distribution of these groups is balanced across the dataset. The dataset was first introduced in scientific literature in work [4].

MBD. is a multimodal banking dataset introduced in [14]. The dataset contains an industrial-scale number of sequences, with data from more than 1.5 million clients. Each client corresponds to a sequence of events. This multi-modal dataset includes card transactions, geo-position events, and embeddings of dialogs with technical support. The goal is to predict the purchases of four banking products in each month, given the historical data from the previous month. For our analysis, we use only card transactions.

Since we focused on the event sequence classification task, we restricted our setup as follows. To predict the purchases, we use transactions from the preceding month. For example, we use a sequence from June to predict a label by the last day of July. We did not use out-of-time validation, as the labeled time span of the data is less than a year. The authors of the dataset split the data into 5 folds (0–4), we use fold 4 as the test fold.

Taobao. Dataset comprises user behaviors from Taobao, including clicks, purchases, adding items to the shopping cart, and favoriting items. These events were collected between November 18 and December 15. For our analysis, we treat each week of clicks as a sequence and aim to predict payments for the subsequent 7 days following the selected week. The training set encompasses data from November 18 to December 1, while the test set includes clicks from December 2 to December 15.

BPI17. We took the dataset from the Business Process Intelligence 2017 Challenge. The dataset describes the logs of events related to the business process of negotiating a loan with the customer. The target is to predict whether the loan offer will be accepted by the customer. For this dataset, we did an out-of-time test split.

ArabicDigits. This data set is taken from the UCI repository [5]. It is derived from sound. Dataset from 8800 (10 digits x 10 repetitions x 88 speakers) time series of 13 Frequency Cepstral Coefficients (MFCCs) had taken from 44 males and 44 females Arabic native speakers between the ages 18 and 40 to represent ten spoken Arabic

[2] https://physionet.org/content/challenge-2012/1.0.0/
[3] https://physionet.org/content/mimiciii/1.4/
[4] https://ods.ai/competitions/sberbank-sirius-lesson
[5] https://ods.ai/competitions/x5-retailhero-uplift-modeling

digit. Each line on the data base represents 13 MFCCs coefficients in the increasing order separated by spaces. This corresponds to one analysis frame.

ElectricDevices. These problems were taken from data recorded as part of government sponsored study called Powering the Nation. The intention was to collect behavioural data about how consumers use electricity within the home to help reduce the UK's carbon footprint. The data contains readings from 251 households, sampled in two-minute intervals over a month. The data required considerable preprocessing to get into a usable format. We create two distinct types of problem: problems with similar usage patterns (Refrigeration, Computers, Screen) and problems with dissimilar usage patterns (Small Kitchen and Large Kitchen). The aim is that problems with dissimilar usage patterns should be well suited to time-domain classification, whilst those with similar consumption patterns should be much harder.

Pendulum. Inspired by [30] we created a pendulum dataset to evaluate time-dependent models. The Pendulum dataset is specifically designed for event sequence classification tasks, featuring irregular timestamps and missing values. Its task requires models to consider multiple events for predictions, making it effective in evaluating temporal modelling capabilities.

The dataset simulates damped pendulum motion with varying lengths. Observation times are sampled irregularly using a Hawkes process, emphasizing the importance of accurate event timing for real-world applications. Each sequence in the dataset consists of events represented by time and two normalized coordinates (x, y), with some values randomly dropped. The goal is to predict the damping factor. We publish the reproducible code to generate the dataset.

To model the Hawkes process, we consider the following intensity function $\lambda(t)$ that is given by (1).

$$\lambda(t) = \mu + \sum_{t_i < t} \alpha e^{-\beta(t - t_i)} \tag{1}$$

We used following parameters for the Hawkes process:

- μ is the base intensity;
- α is the excitation factor, was chosen to be 0.5;
- β is the decay factor, was set to 1.
- t_i are the times of previous events before time t.

The time points are sampled within the interval $[0, \text{end time}]$, where the end time is sampled from a uniform distribution $U(3, 5)$. To maintain an approximately constant number of points (30) per sequence, we adjust the base intensity μ as follows:

$$\mu = 30 \times \frac{1 - \alpha}{\text{end time} - 1}$$

This ensures each sequence has a dynamic time interval but approximately the same number of points, preventing the model from learning the timestamp distribution without using timestamp data.

To model the pendulum we consider the second-order differential equation:

$$\theta'' + \left(\frac{b}{m}\right)\theta' + \left(\frac{g}{L}\right)\sin(\theta) = 0 \tag{2}$$

where,

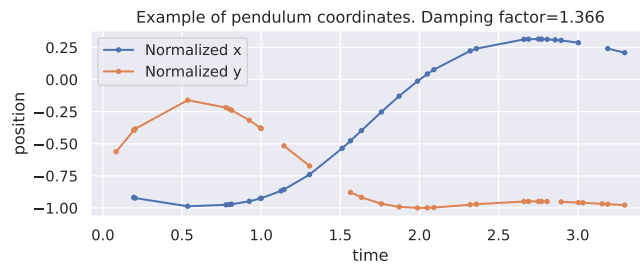


Figure 4: Pendulum motion at various instances, with time steps determined by a Hawkes process.

- θ'' is the Angular Acceleration,
- θ' is the Angular Velocity,
- θ is the Angular Displacement,
- b is the Damping Factor,
- $g = 9.81\,\text{m/s}^2$ is the acceleration due to gravity,
- L is the Length of pendulum,
- m is the Mass of bob in kg.

To convert this second-order differential equation into two first-order differential equations, we let $\theta_1 = \theta$ and $\theta_2 = \theta'$, which gives us:

$$\theta_2' = \theta'' = -\left(\frac{b}{m}\right)\theta_2 - \left(\frac{g}{L}\right)\sin(\theta_1) \tag{3}$$

$$\theta_1' = \theta_2 \tag{4}$$

Thus, the first-order differential equations for the pendulum simulation are:

$$\theta_2' = -\left(\frac{b}{m}\right)\theta_2 - \left(\frac{g}{L}\right)\sin(\theta_1) \tag{5}$$

$$\theta_1' = \theta_2 \tag{6}$$

In our simulations, the damping factor b is sampled from a uniform distribution $U(1, 3)$, and the mass of the bob $m = 1$. The length L of the pendulum is taken from a uniform distribution $U(0.5, 10)$, representing a range of possible lengths from 0.5 to 10 meters. The initial angular displacement θ is sampled from a uniform distribution $U(0, 2\pi)$, and the initial angular velocity θ' is sampled from a uniform distribution $U(-\pi, \pi)$, providing a range of initial conditions in radians and radians per second, respectively.

Our primary objective is to predict the damping factor b, using the normalized coordinates x and y on the plane. These coordinates are scaled with respect to the pendulum's length, such that the trajectory of the pendulum is represented in a unitless fashion. This normalization allows us to abstract the pendulum's motion from its actual physical dimensions and instead focus on the pattern of movement. Additionally, we randomly drop 10% of values for both coordinates. An illustrative example of this motion is presented in Figure 4.

C HPO details

Hyperparameter Optimization (HPO) is a critical step in the development and evaluation of machine learning models. It involves systematically searching for the optimal set of hyperparameters that maximize model performance. In this section, we outline our

main evaluation methodology and HPO process, which is detailed in Algorithm 1.

Our approach includes two main steps: the HPO step and the final evaluation step. In the HPO step, we use the Tree-structured Parzen Estimator (TPE) to efficiently search the hyperparameter space. We split the training dataset into three subsets: `train` (70%), `train-val` (15%), and `hpo-val` (15%). The model is trained on the `train` set, and its performance is evaluated on the `train-val` set to determine when to stop training. The `hpo-val` set is used to update the TPE sampler and guide the selection of hyperparameters.

After the HPO step, we proceed to the final evaluation step. Here, we use the best hyperparameters (BHP) identified in the HPO step to train and evaluate the model multiple times with different random seeds. This ensures that our results are robust and not dependent on a particular random initialization. The training dataset is split into `train` (85%) and `train-val` (15%) sets, and the model is trained until performance on the `train-val` set stops improving or until the training budget is exhausted. Finally, we evaluate the model on the test set and report the mean and standard deviation of the test metrics.

For more details about the HPO process, we refer to our Algorithm 1.

D Number of learnable parameters

We report number of learnable parameters of each model from Table 2 in Table6

E Subsets metric relationships

Algorithm 1 Our main evaluation methodolgy and HPO, here N_{hpo} - is HPO budget, $MaxIters$ - training budget, N_{seeds} - a number of iterations for random seed runs.

```
MaxIters = 10^5
N_seeds = 20
start HPO step
split train dataset randomly into three subsets train (70%), train-val (15%) and hpo-val (15%)
initalize TPE
for i = 1, 2, ..., N_hpo do
    set model hyper parameters with TPE
    train a model until performance on train-val set stops improving or until we run out from the budget MaxIters.
    update TPE sampler using metrics obtained on hpo-val
end for
select best hyper parameters (BHP) according to hpo-val metrics
Start Final evaluation step
for seed = 1, 2, ..., N_seeds do
    set a new random seed
    randomly split train dataset into train (85%) and train-val (15%) sets
    train a model with BHP until performance on train-val set stops improving or until we run out from the budget MaxIters.
    evaluate the model on test set
end for
Report mean and std of test metrics from Final evaluation step
```

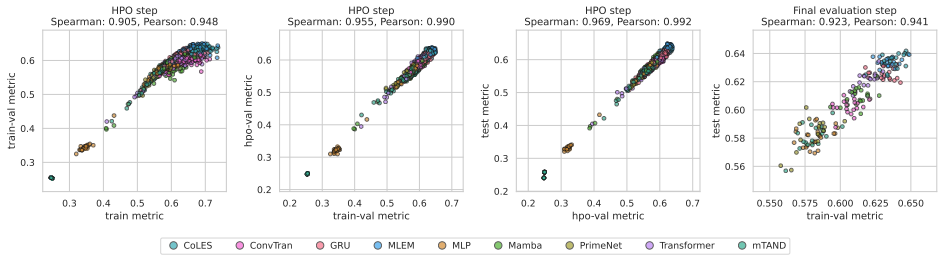


Figure 5: Performance metric relationships and correlations of different subsets among all methods on Age dataset

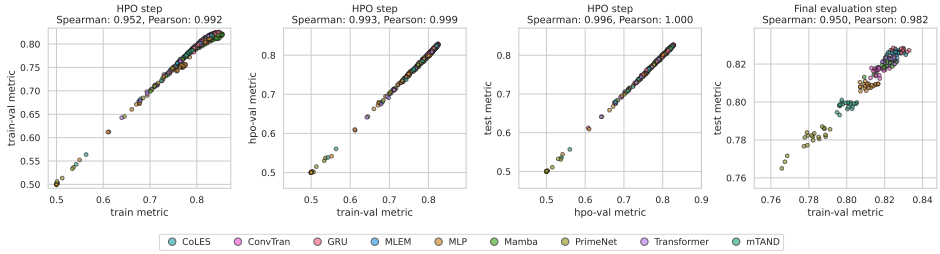


Figure 6: Performance metric relationships and correlations of different subsets among all methods on MBD dataset

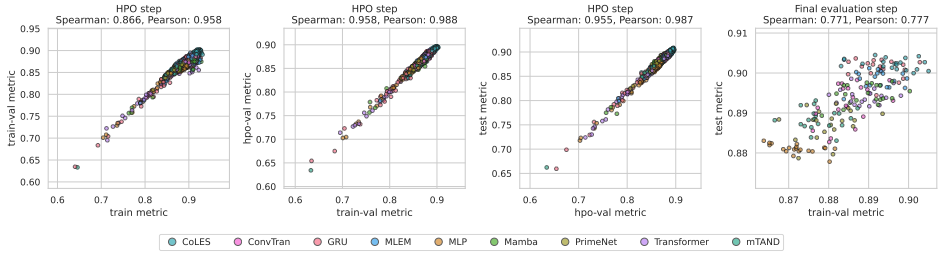


Figure 7: Performance metric relationships and correlations of different subsets among all methods on MIMIC-III dataset

Table 6: Number of learnable parameters for each model from Table 2. Based on successful HPO runs, we report the minimum and maximum number of parameters explored in the search space, along with the number of parameters in the best-performing model. The results indicate that most models do not select the heaviest settings, likely because excessively increasing model size leads to overfitting on the EvS classification task.

Method	Params	CoLES	GRU	MLEM	Transformer	Mamba	ConvTran	mTAND	PrimeNet	MLP
MBD	**min**	2.1e+05	3.2e+05	9.0e+06	1.9e+06	4.6e+05	1.5e+05	3.8e+05	3.5e+05	4.6e+05
	best	9.1e+06	1.1e+07	1.4e+07	1.4e+08	5.2e+07	3.8e+06	5.1e+06	1.7e+06	6.4e+06
	max	2.9e+07	3.0e+07	5.1e+07	3.0e+08	5.7e+07	1.7e+07	1.2e+07	4.2e+06	7.9e+06
Retail	**min**	1.2e+05	8.8e+04	1.6e+06	2.5e+06	2.1e+05	8.1e+04	1.4e+04	3.7e+04	2.3e+04
	best	2.3e+06	3.8e+06	6.1e+06	2.8e+08	2.2e+07	1.7e+06	1.0e+05	9.6e+04	3.5e+05
	max	2.7e+07	2.4e+07	3.5e+07	3.1e+08	9.0e+07	5.1e+06	7.8e+06	1.8e+06	3.1e+06
Age	**min**	2.7e+04	1.0e+04	4.2e+05	3.3e+04	2.4e+04	2.7e+04	5.3e+03	1.5e+04	3.5e+03
	best	4.3e+05	1.2e+05	9.0e+06	1.0e+06	3.6e+05	2.9e+06	2.7e+04	3.9e+04	5.2e+05
	max	1.9e+07	1.1e+07	1.3e+07	1.6e+07	2.9e+06	3.5e+06	6.6e+06	2.2e+06	1.6e+06
Taobao	**min**	7.6e+04	7.8e+03	3.0e+06	1.7e+04	1.2e+04	9.9e+03	5.5e+04	5.4e+04	3.6e+03
	best	4.0e+06	1.0e+05	7.4e+06	1.5e+06	1.2e+06	1.4e+06	4.1e+06	1.3e+06	1.1e+05
	max	2.2e+07	1.6e+07	3.1e+07	1.6e+07	3.1e+06	3.5e+06	1.2e+07	2.5e+06	1.6e+06
BPI17	**min**	9.3e+04	2.7e+04	8.4e+05	1.4e+06	7.1e+04	8.6e+04	2.7e+04	1.2e+04	7.3e+03
	best	6.1e+05	2.1e+05	5.0e+06	1.6e+08	1.7e+06	4.4e+06	1.5e+05	1.3e+05	1.6e+05
	max	1.7e+07	2.5e+07	2.2e+07	3.5e+08	6.1e+07	1.7e+07	1.0e+07	3.7e+06	3.5e+06
PhysioNet2012	**min**	3.6e+04	1.7e+04	8.1e+05	1.0e+07	7.8e+04	2.5e+04	1.3e+04	1.3e+04	5.8e+03
	best	4.0e+05	2.3e+06	1.7e+06	3.1e+08	3.5e+07	5.4e+06	7.6e+05	6.6e+04	4.7e+04
	max	2.7e+07	3.3e+07	3.5e+07	1.8e+09	5.4e+08	2.2e+07	1.1e+07	3.1e+06	5.3e+06
MIMIC-III	**min**	8.2e+04	7.9e+03	5.6e+05	4.5e+05	8.7e+03	6.3e+03	5.2e+03	1.5e+04	5.5e+03
	best	2.4e+05	1.1e+06	1.3e+07	2.2e+07	8.0e+05	2.6e+06	4.9e+04	1.5e+06	1.0e+06
	max	2.2e+07	1.7e+07	3.3e+07	2.5e+08	4.5e+07	1.4e+07	9.6e+06	2.8e+06	2.9e+06
Pendulum	**min**	2.5e+04	6.3e+03	2.6e+05	2.0e+04	3.3e+03	3.4e+03	8.0e+03	1.6e+04	7.8e+02
	best	3.3e+05	1.3e+05	6.5e+06	5.9e+06	2.6e+06	6.2e+05	1.0e+06	1.2e+06	3.9e+04
	max	2.2e+07	2.2e+07	1.8e+07	1.6e+07	2.9e+06	6.4e+06	1.1e+07	2.7e+06	1.8e+06
ArabicDigits	**min**	3.8e+04	3.9e+03	3.7e+06	8.9e+04	1.6e+04	1.5e+05	1.8e+04	3.5e+04	3.3e+03
	best	5.5e+06	5.3e+05	1.1e+07	2.5e+05	5.7e+07	4.8e+06	2.1e+05	6.5e+05	3.0e+06
	max	2.4e+07	2.5e+07	3.7e+07	3.5e+08	6.1e+07	3.0e+07	6.7e+06	3.9e+06	3.5e+06
ElectricDevices	**min**	2.5e+04	7.2e+03	2.4e+06	3.8e+04	8.0e+02	5.1e+03	1.3e+04	2.1e+03	4.4e+02
	best	1.2e+07	3.6e+05	1.7e+07	3.4e+05	2.9e+05	2.0e+06	4.4e+04	9.6e+04	7.3e+05
	max	2.2e+07	2.2e+07	4.0e+07	7.1e+06	1.3e+06	4.3e+06	8.2e+06	2.1e+06	1.3e+06

Table 7: Including vs. Excluding time as a feature. We compare the top 5 test metrics from the HPO step for each option and report the relative performance difference of metrics obtained excluding time feature to metrics obtained including time as a feature. MLEM not included since it has fixed time process option - copied from best CoLES.

Category	Discrete **EvS**					Continuous **EvS**			Time Series	
Dataset	**MBD**	**Retail**	**Age**	**Taobao**	**BPI17**	**PhysioNet2012**	**MIMIC-III**	**Pendulum**	**ArabicDigits**	**ElectricDevices**
Metric	Mean ROC AUC	Accuracy	Accuracy	ROC AUC	ROC AUC	ROC AUC	ROC AUC	Accuracy	Accuracy	Accuracy
CoLES	−0.99%	0.18%	−0.31%	−1.32%	−0.16%	−0.41%	−0.46%	−61.34%	−0.38%	−1.75%
GRU	−1.17%	0.12%	−0.82%	−1.93%	−0.34%	−0.27%	−0.43%	−60.40%	−0.08%	−0.55%
Transformer	−1.36%	−2.08%	−0.84%	−1.22%	0.40%	−0.84%	−0.88%	−56.99%	−0.10%	0.82%
Mamba	−0.99%	−0.02%	−0.65%	−1.65%	0.53%	−0.57%	−0.57%	−59.31%	−0.13%	−0.73%
ConvTran	−0.62%	−0.57%	−0.53%	−0.09%	−0.16%	−0.39%	−0.49%	−59.16%	−0.09%	−3.84%
mTAND	−9.67%	−0.42%	−0.63%	−3.14%	−0.22%	−0.38%	−0.81%	−22.87%	−1.37%	0.08%
PrimeNet	−4.62%	−1.03%	−1.02%	−4.45%	−0.51%	−0.09%	−0.54%	−14.26%	−0.68%	−2.78%
MLP	−1.03%	−0.16%	−0.98%	−6.44%	0.00%	−0.12%	−1.18%	−10.72%	−6.91%	−1.54%

Table 8: Different aggregation approaches: mean across all hidden states or last hidden state. We take top 5 test metrics from HPO step for each option and report mean and std. Highlighted bold if adding time significantly improves performance.

Dataset	Aggregation	CoLES	GRU	MLEM	Transformer	Mamba	mTAND	MLP
MBD	**Last hidden**	**0.825 ± 0.000**	**0.826 ± 0.000**	**0.823 ± 0.001**	0.818 ± 0.001	0.822 ± 0.001	**0.795 ± 0.002**	0.755 ± 0.001
	Mean hidden	0.820 ± 0.002	0.822 ± 0.001	0.816 ± 0.006	**0.822 ± 0.001**	0.822 ± 0.001	0.786 ± 0.002	**0.809 ± 0.000**
Retail	**Last hidden**	**0.551 ± 0.000**	**0.544 ± 0.001**	**0.546 ± 0.001**	0.537 ± 0.001	0.528 ± 0.001	0.518 ± 0.001	0.343 ± 0.000
	Mean hidden	0.547 ± 0.001	0.542 ± 0.001	0.539 ± 0.002	0.541 ± 0.002	**0.540 ± 0.000**	0.519 ± 0.001	**0.526 ± 0.000**
Age	**Last hidden**	**0.641 ± 0.001**	0.620 ± 0.001	**0.647 ± 0.001**	0.603 ± 0.004	0.588 ± 0.007	**0.589 ± 0.002**	0.338 ± 0.002
	Mean hidden	0.638 ± 0.001	**0.630 ± 0.001**	0.635 ± 0.002	**0.624 ± 0.002**	**0.620 ± 0.002**	0.582 ± 0.004	**0.597 ± 0.004**
Taobao	**Last hidden**	**0.718 ± 0.001**	**0.718 ± 0.001**	**0.718 ± 0.000**	0.712 ± 0.001	0.680 ± 0.012	**0.680 ± 0.001**	0.603 ± 0.023
	Mean hidden	0.710 ± 0.001	0.713 ± 0.001	0.709 ± 0.001	0.712 ± 0.001	**0.710 ± 0.002**	0.676 ± 0.001	**0.685 ± 0.001**
BPI17	**Last hidden**	**0.759 ± 0.002**	**0.760 ± 0.001**	**0.762 ± 0.001**	0.758 ± 0.001	0.741 ± 0.002	0.733 ± 0.002	0.709 ± 0.004
	Mean hidden	0.737 ± 0.006	0.755 ± 0.002	0.747 ± 0.002	0.757 ± 0.002	**0.749 ± 0.002**	**0.740 ± 0.002**	**0.738 ± 0.001**
PhysioNet2012	**Last hidden**	**0.846 ± 0.001**	**0.848 ± 0.001**	**0.849 ± 0.002**	**0.843 ± 0.001**	**0.840 ± 0.002**	0.844 ± 0.000	**0.845 ± 0.001**
	Mean hidden	0.827 ± 0.002	0.818 ± 0.003	0.831 ± 0.003	0.836 ± 0.005	0.809 ± 0.007	0.845 ± 0.001	0.815 ± 0.003
MIMIC-III	**Last hidden**	**0.908 ± 0.001**	**0.901 ± 0.001**	**0.899 ± 0.000**	**0.894 ± 0.001**	0.887 ± 0.003	**0.891 ± 0.002**	**0.880 ± 0.000**
	Mean hidden	0.898 ± 0.001	0.894 ± 0.001	0.896 ± 0.001	0.890 ± 0.001	**0.898 ± 0.001**	0.886 ± 0.001	0.874 ± 0.001
Pendulum	**Last hidden**	0.725 ± 0.007	0.671 ± 0.011	0.677 ± 0.008	**0.633 ± 0.007**	0.640 ± 0.005	**0.780 ± 0.012**	**0.194 ± 0.000**
	Mean hidden	0.725 ± 0.002	**0.703 ± 0.009**	0.665 ± 0.005	0.580 ± 0.022	**0.681 ± 0.002**	0.711 ± 0.007	0.158 ± 0.002
ArabicDigits	**Last hidden**	0.992 ± 0.001	0.991 ± 0.002	0.994 ± 0.001	0.985 ± 0.001	0.989 ± 0.001	0.964 ± 0.002	0.477 ± 0.007
	Mean hidden	0.990 ± 0.002	0.992 ± 0.001	0.995 ± 0.000	**0.991 ± 0.000**	**0.993 ± 0.001**	0.958 ± 0.005	**0.775 ± 0.001**
ElectricDevices	**Last hidden**	**0.758 ± 0.005**	0.752 ± 0.002	**0.749 ± 0.003**	0.728 ± 0.007	0.729 ± 0.004	0.640 ± 0.005	**0.465 ± 0.001**
	Mean hidden	0.737 ± 0.004	0.749 ± 0.003	0.732 ± 0.004	0.731 ± 0.009	**0.750 ± 0.003**	0.640 ± 0.011	0.254 ± 0.000

Table 9: Different normalization approaches: with vs without Batch Normalization for input features. We compare the top 5 test metrics from the HPO step for each option and report the relative performance difference of metrics obtained without batch normalization to metrics obtained with batch normalization.

Category	Discrete **EvS**					Continuous **EvS**			Time Series	
Dataset	**MBD**	**Retail**	**Age**	**Taobao**	**BPI17**	**PhysioNet2012**	**MIMIC-III**	**Pendulum**	**ArabicDigits**	**ElectricDevices**
Metric	Mean ROC AUC	Accuracy	Accuracy	ROC AUC	ROC AUC	ROC AUC	ROC AUC	Accuracy	Accuracy	Accuracy
CoLES	−0.25%	−3.30%	−0.68%	−1.36%	−3.53%	−7.28%	−2.54%	11.78%	−0.06%	−0.46%
GRU	−0.57%	−4.01%	−0.30%	−1.32%	−3.83%	−7.75%	−2.53%	10.94%	0.06%	−0.42%
MLEM	5.82%	−2.95%	0.30%	−1.21%	−4.14%	−12.46%	−2.39%	16.94%	0.15%	−1.36%
Transformer	0.00%	−21.29%	−0.10%	−0.68%	−5.59%	−6.65%	−2.38%	15.07%	−0.25%	−1.47%
Mamba	−0.46%	−3.66%	−0.61%	−1.27%	−1.87%	−0.73%	−1.38%	10.19%	0.46%	1.05%
ConvTran	0.07%	0.19%	−2.31%	0.07%	0.23%	−2.48%	−0.61%	9.47%	0.19%	−2.66%
mTAND	1.66%	−0.98%	−0.75%	−3.61%	−0.93%	−4.47%	−1.46%	17.35%	0.89%	4.24%
PrimeNet	−0.04%	−1.04%	0.30%	−4.27%	−1.08%	−1.73%	0.09%	19.08%	−0.91%	−2.87%
MLP	−0.40%	−3.00%	−1.07%	−7.75%	−1.03%	−4.07%	−3.87%	5.75%	−1.84%	−0.18%

Table 10: Learning Rate Importance by Optuna Ranking (Smaller Rank = Higher Importance). There is a unique best Learning Rate for each Dataset/Method combination

Category	Discrete **EvS**					Continuous **EvS**			Time Series	
Dataset	**MBD**	**Retail**	**Age**	**Taobao**	**BPI17**	**PhysioNet2012**	**MIMIC-III**	**Pendulum**	**ArabicDigits**	**ElectricDevices**
Metric	Mean ROC AUC	Accuracy	Accuracy	ROC AUC	ROC AUC	ROC AUC	ROC AUC	Accuracy	Accuracy	Accuracy
CoLES	9	4	1	1	2	3	2	2	2	1
GRU	1	1	2	1	1	4	2	4	1	1
MLEM	3	1	2	1	1	4	4	9	1	1
Transformer	4	8	1	3	3	7	6	7	5	1
Mamba	1	1	1	1	1	1	1	1	1	1
ConvTran	5	1	9	2	1	1	1	3	5	2
mTAND	1	1	1	1	1	3	1	1	1	1
PrimeNet	1	1	5	1	1	2	1	1	1	7
MLP	1	1	2	2	1	1	3	1	2	2

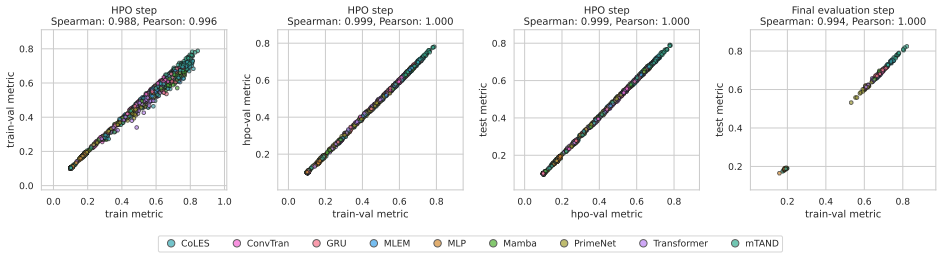


Figure 8: Performance metric relationships and correlations of different subsets among all methods on Pendulum dataset

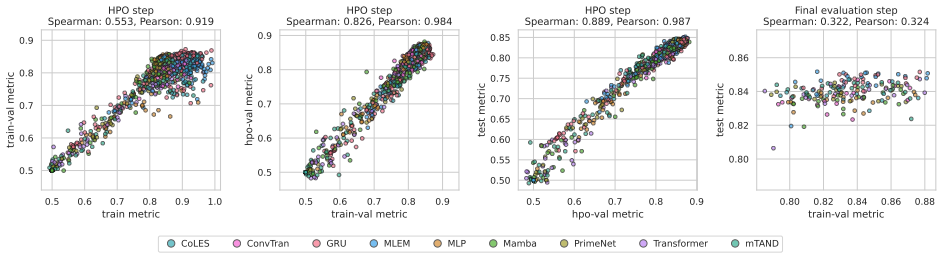


Figure 9: Performance metric relationships and correlations of different subsets among all methods on PhysioNet2012 dataset

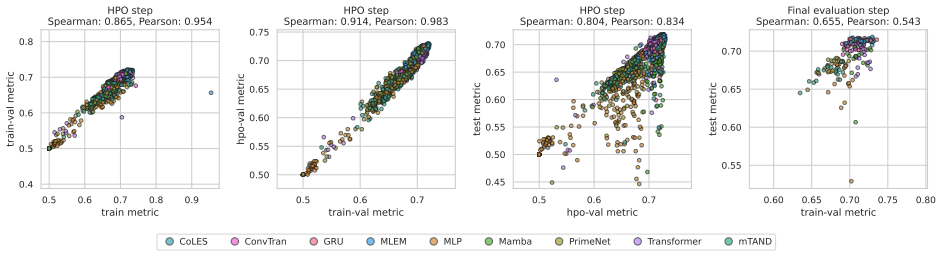


Figure 10: Performance metric relationships and correlations of different subsets among all methods on Taobao dataset

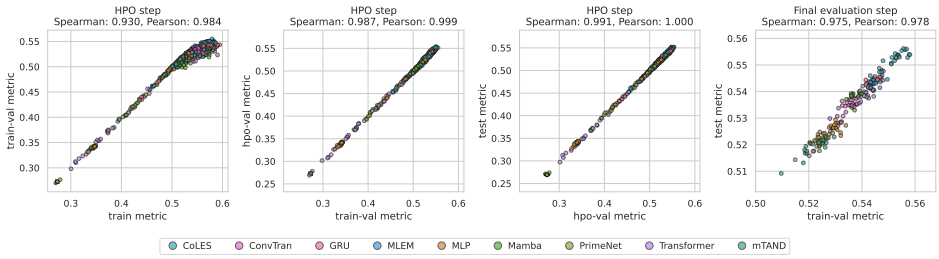


Figure 11: Performance metric relationships and correlations of different subsets among all methods on Retail dataset

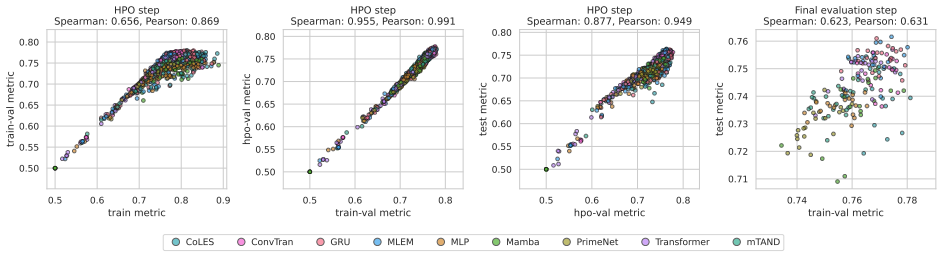


Figure 12: Performance metric relationships and correlations of different subsets among all methods on BPI17 dataset

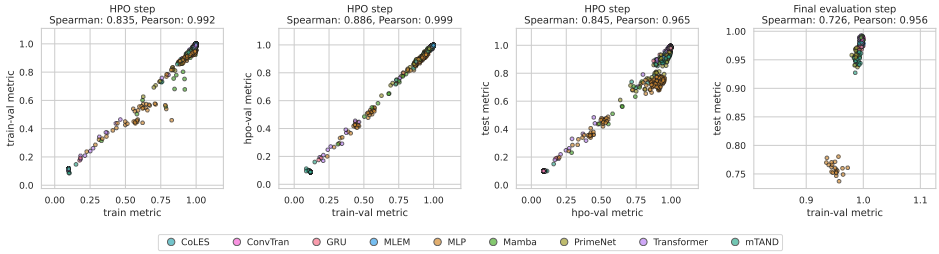


Figure 13: Performance metric relationships and correlations of different subsets among all methods on ArabicDigits dataset

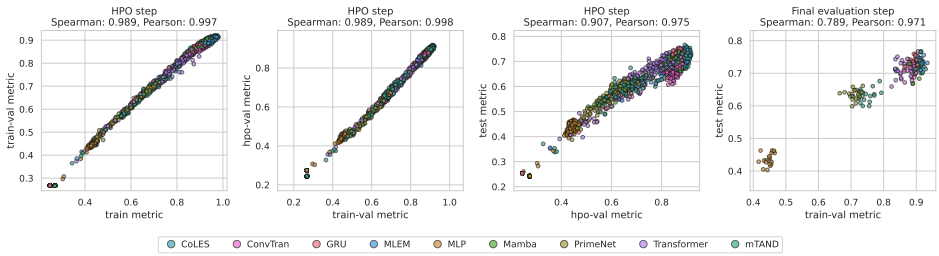


Figure 14: Performance metric relationships and correlations of different subsets among all methods on ElectricDevices dataset